
Ch09

01. 참조

02. 포인터

03. 배열과 포인터

04. 메모리

참조

- 참조(reference)는 객체의 또 다른 이름(별칭, alias)
- 참조는 C++에서 추가된 기능으로, 함수 등의 엔티티끼리 커뮤니케이션하는 것을 쉽게 만들어줌, **Pass by reference**

소개(1)

- 참조 변수 reference variable 는 일반적인 변수와 다르게 메모리 위에 새로운 영역을 잡지 않음
- 단순하게 기존에 있던 변수와 연결(바인드)해서, 기존에 있던 변수에 새로운 이름을 붙여주는 것뿐
- 따라서 원본 변수가 있어야만 생성할 수 있음
- 다음은 int 자료형의 변수 score를 만들고, 이를 기반으로 참조 자료형의 변수 rScore를 만든 뒤 연결하는 코드

```
int score = 92;           // Declaring and initializing score
int& rScore = score;      // 선언하면서, 변수와 바인딩 (and 초기화)

Int& x = 92;              // Error: no binding to literal. 반드시 변수에 binding
```

소개(3)

참조 자료형

- 참조 자료형은 복합 자료형으로 int의 참조(int&), double의 참조(double&), bool의 참조(bool&) 형태로 구성
- 변수를 참조 변수와 연결할 때는 반드시 ‘변수의 자료형’과 ‘참조 변수가 나타내는 자료형’이 일치해야 함
- 예를 들어서 다음과 같은 코드는 오류가 발생

```
int num = 100;  
double& rNum = num; // Compilation error. Type mismatch
```

소개(4)

참조 관계

- 참조 변수가 선언되고 어떤 변수와 연결되면, 참조 관계는 스코프를 벗어나서 변수가 파괴되기 전까지는 계속해서 유지
- 즉 참조 관계는 한 번 정의되면 변경할 수 없음
- C++에서는 변수와 참조 변수가 연결된 관계를 **영속적인 관계** **constant relation**이라고 표현
- 이 책에서는 쉽게 이해할 수 있게 ‘영구적인 관계’라고 표현

소개(6)

예시

- 다음 코드는 rScore 참조 변수를 score와 연결한 뒤, 이를 끊고 num과 연결하는 예시
- 참조 관계를 변경하려고 할 때 컴파일 오류가 발생

```
int score = 92;  
int& rScore = score;  
int num = 80;  
int& rScore = num; // error, breaking the reference relationship
```

constant relation

소개(7)

예시

```
int score = 92;  
int& rScore = score;    value 가 바뀌는 대입문  
int num = 80;           constant relation 을 깨는 것이 아님!!  
rScore = num; // 단순한 assignment 문
```

- 이 코드의 마지막 문장은 참조 관계를 깨는 것이 아님
- 공유하고 있는 메모리 위치에 num의 값을 넣는 것이지, 참조 연결을 하는 문장이 아님
- 참고로 rScore는 score의 별칭이므로, rScore = num은 score = num과 같은 기능을 함

값 추출(1)

- 참조 관계가 만들어지면, 원본 변수와 참조 변수 중에 아무 것이나 사용해도 값에 접근하여 값을 추출할 수 있음
- [프로그램 9-1]은 원본 변수와 참조 변수를 사용하는 예시
- 이 프로그램은 개념 이해를 위한 예시이며, 참조 변수는 이번 강의 후반부에서 설명하는 것처럼 함수와 함수의 커뮤니케이션에 사용하는 것이 일반적

값 추출(2)

프로그램 9-1 값에 접근하기

```
1  /*****
2  * The program shows how to declare and initialize the original*
3  *and reference variables and then access the common value      *
4  * through either of them.                                     *
5  *****/
6  #include <iostream>
7  using namespace std;
8
9  int main ( )
10 {
11     // Creation of reference relations
12     int score = 92;
13     int& rScore = score;
14     // Using data variable
15     cout << "Accessing value through data variable." << endl;
16     cout << "score: " << score << endl;
17     // Using reference variable
18     cout << "Accessing value through reference variable." << endl;
19     cout << "rScore: " << rScore;
20     return 0;
```

값 추출(3)

프로그램 9-1 값에 접근하기

```
21 }
```

Run:

Accessing value through data variable.

score: 92

Accessing value through reference variable.

rScore: 92

값 변경(3)

표 9-1 const 한정자를 붙이는 조합

경우	데이터 변수	참조 변수	상태
1	int name = value;	int& rName = name;	가능
2	const int name = value;	int& rName = name;	오류
3	int name = value;	const int& rName = name;	가능
4	const int name = value;	const int& rName = name;	가능

경우1

- 원본 변수와 참조 변수 중에서 아무 것이나 사용해도 메모리 위치의 값을 변경할 수 있음

```
int name = 5; int& rName = name; rName = 55; // ok
```

경우3

- 원본 변수를 사용해서는 값을 변경할 수 있지만, 참조 변수를 사용해서는 값을 변경할 수 없음.
원본변수의 값을 변경시키지 않기를 위함

```
int name = 5; const int& rName = name; name = 55; //ok
```

```
rName = 55; // error
```

참조의 활용(1)

참조로 전달

- 이번에는 두 함수의 커뮤니케이션에서 참조를 활용하는 방법을 다루며 함수의 커뮤니케이션이란 데이터를 전달하고 리턴하는 것
- 참조를 활용해서 데이터를 전달하는 것을 **pass-by-reference**, 참조를 활용해서 데이터를 리턴하는 것을 **return-by-reference**라고 부름



그림 9-6 값으로 전달과 참조로 전달 비교

참조의 활용(2)

- 각각의 방법으로 전달된 정수를 처리하는 프로그램을 만들면 다음과 같음

```
#include <iostream>

using namespace std;
// Prototype
void doIt(int);

int main()
{
    int num = 10;

    doIt(num);
    return 0;
}
// Pass-by-value
void doIt(int num)
{
    num++;
}
```

```
#include <iostream>

using namespace std;

// Prototype
void doIt(int&);

int main()
{
    int num = 10;
    doIt(num);
    return 0;
} 함수 종료시 relationship 파괴
// Pass-by-reference

void doIt(int& rNum)
{
    rNum++;
}
```

참조의 활용(3)

Pass-by-value의 특징

- 값으로 전달은 호출하는 함수 쪽의 값(argument)이 호출되는 함수 쪽의 변수(parameter)로 복사
- 내부적으로 다음과 같은 문장이 실행된다고 할 수 있음

```
int num2 = num1;
```

- 값으로 전달에서는 인수 argument 와 매개변수 parameter 가 독립적으로 존재
- 따라서 매개변수를 변경하는 일이 인수를 변경하는 일에 영향을 주지 않음
- 이는 어떤 상황에서는 장점이며, 어떤 상황에서는 단점이 됨

참조의 활용(4)

- 값으로 전달에서 문제가 되는 또 다른 부분은 복사 비용
- 복사 대상 객체가 크다면, 인수를 매개변수로 복사할 때 많은 복사 비용이 생성
- 기본 자료형 등의 크기가 작은 자료형을 복사할 때는 큰 문제가 없지만, 클래스 자료형처럼 크기가 큰 자료형을 복사할 때는 다른 방법을 고려하는 것이 좋음

참조의 활용(5)

pass-by-reference의 특징

- 참조 전달을 사용하면 함수에서 내부적으로 다음과 같이 연결

```
int num2 = num1;
```

- 참조로 전달에서 인수와 매개변수는 동일한 객체입니다. 따라서 메모리를 추가로 할당하지 않음
- 참조로 전달은 인수와 매개변수가 동일한 객체이므로 따로 복사할 필요가 없기 때문에 복사 비용이 발생하지 않으며 클래스 자료형처럼 큰 객체를 전달할 때는 참조로 전달을 우선적으로 고려하는 것이 좋음

참조의 활용(6)

주의

- 다만 참조로 전달을 사용할 때는 매개변수에 값을 지정할 수 없다는 점을 유의
- 다음과 같은 코드는 오류를 발생시킴

```
void fun(int& rX) { ... } // Function definition
fun(5); // Function call (compilation error). argument는 반드시 변수여야함
```

- 매개변수 rX가 참조 매개변수이므로, 함수를 호출할 때는 변수를 argument로 사용하여야 함

참조의 활용(7)

예시

- 기본 자료형의 값을 매개변수로 받아서 출력하는 함수를 만든다고 가정
- 출력만 하는 것이므로 전달된 값을 따로 변경할 필요도 없음
- 값으로 전달과 `const` 한정자를 붙인 참조로 전달을 모두 사용할 수 있음
- 값이 작으므로 값으로 전달을 사용해도 됨

SKIP - 참조의 활용(8)

예시

- 클래스의 복사 생성자를 만드는 경우는 어떨까?
- 객체를 전달해야 하므로 값으로 전달은 복사 비용이 많이 들 수 있고, 값으로 전달 자체가 복사 생성자를 활용하는 것이므로 복사 생성자에서는 값으로 전달을 사용할 수 없음(무한 반복에 빠짐)
- 따라서 `const` 한정자를 붙인 참조를 사용해야 함

SKIP - 참조의 활용(9)

- C++ 표준에서도 다음과 같이 사용하라고 나와 있음

// Pass-by-reference in copy constructor

```
Circle :: Circle(const Circle& circle)
: radius(circle.radius)
{
}
```

예시

- 두 데이터 요소를 교체하는 함수를 생성
- [프로그램 9-2]처럼 참조로 전달을 사용하게 코드를 변경

참조의 활용(10)

프로그램 9-2 참조로 전달을 사용해서 스왑 함수 만들기

```
1  /*****
2  * The program shows how to use pass-by-reference to allow a *
3  * called function to swap two values in the calling function.*
4  *****/
5  #include <iostream>
6  using namespace std;
7
8  void swap (int& first, int& second) ; // Prototype
9
10 int main ( )
11 {
12     // Definition of two variables
13     int x = 10;
14     int y = 20;
15     // Printing the value of x and y before swapping
16     cout << "Values of x and y before swapping." << endl;
17     cout << "x: " << x << " " << "y: " << y << endl;
18     // Calling swap function to swap the values of x and y
19     swap (x , y);
20     // Printing the value of x and y after swapping
```

참조의 활용(11)

프로그램 9-2 참조로 전달을 사용해서 스왑 함수 만들기

```
21     cout << "Values of x and y after swapping." << endl;
22     cout << "x: " << x << " " << "y: " << y;
23     return 0;
24 }
25 /*****
26 * The swap function swaps the values of the parameters and      *
27 * pass-by-reference allows the corresponding arguments in main *
28 * to be swapped accordingly.                                     *
29 *****/
30 void swap (int& rX, int& rY)
31 {
32     int temp = rX;
33     rX = rY;
34     rY = temp;
35 }
```

Run:

Values of x and y before swapping.

x: 10 y: 20

Values of x and y after swapping.

x: 20 y: 10

참조의 활용(12)

참조로 리턴

- 참조로 리턴은 호출된 함수에서 호출한 함수 쪽으로 객체를 리턴할 때 사용

참조의 활용(13)

Return-by-value(값으로 리턴)의 특징

- 값으로 리턴에서는 호출된 함수는 다음과 같은 프로토타입을 갖고 원하는 자료형의 객체를 리턴

```
type function(...);
```

- 값으로 리턴은 굉장히 간단하게 매개변수 또는 지역 변수의 값을 리턴할 수 있음
- 유일한 단점은 복사 비용
- 리턴해야 하는 객체가 기본 자료형이라면 괜찮지만 클래스 자료형이라면 복사 생성자가 호출될 것이며, 일반적으로 복사에 높은 비용이 들어감

참조의 활용(14)

return-by-reference (참조로 리턴)의 특징

- 참조로 리턴에서는 호출된 함수는 다음과 같은 프로토타입을 갖고, 원하는 자료형에 대한 참조를 리턴

```
type& function(...);
```

- 참조로 리턴은 복사 비용이 없지만, 일반적인 매개변수와 지역 변수의 값을 리턴할 수 없음
- 이번 장의 후반부에서 이유를 자세하게 살펴보겠지만, 함수가 종료될 때 모든 일반 매개변수와 지역 변수의 값이 파괴되기 때문
- 객체가 파괴되면 원본 변수가 파괴되므로, 참조 변수가 가리키는 메모리 위치 자체가 사라짐
- 반대로 말하면 참조 매개변수와 정적 지역 변수는 함수가 종료되어도 파괴되지 않으므로 리턴할 수 있음

참조의 활용(15)

예시

- 두 정수 중에 더 큰 정수를 찾는 함수를 만든다고 가정
- 값으로 전달과 값으로 리턴 조합을 사용할 수도 있고, 참조로 전달과 참조로 리턴 조합을 사용할 수도 있을 것
- 다음은 두 프로그램을 모두 비교한 예시

참조의 활용(16)

```
#include <iostream>
using namespace std;
// Return-by-value
int larger(int x, int y)
{
    if (x > y)
    {
        return x;
    }
    return y;
}

int main()
{
    int x = 10;
    int y = 20;
    int z = larger(x, y);
    cout << z;
    return 0;
}
```

```
#include <iostream>
using namespace std;
// Return-by-reference
int& larger(int& x, int& y)
{
    if (x > y)
    {
        return x;
    }
    return y;
}

int main()
{
    int x = 10;
    int y = 20;
    int z = larger(x, y);
    cout << z;
    return 0;
}
```